Logistic Regression

1. Introduction to Logistic Regression

1.1 What is Logistic Regression?

Logistic Regression is a supervised machine learning algorithm primarily used for classification, especially binary classification.

Binary classification means predicting one of two possible classes.

Examples:

- Email: Spam (1) or Not Spam (0)
- Medical prediction: Diabetes (1) or No Diabetes (0)
- Transaction: Fraud (1) or Not Fraud (0)
- Student result: Pass (1) or Fail (0)

1.2 Logistic Regression vs. Linear Regression

Feature	Linear Regression	Logistic Regression
Main purpose	Regression	Classification
Output	Continuous numerical value	Probability between 0 and 1
Example	Predicting salary	Predicting diabetes
Main function	Linear equation	Linear equation + sigmoid
Common loss	Mean Squared Error (MSE)	Binary Cross-Entropy

Linear Regression might predict a salary of 35,000. Logistic Regression might predict a diabetes probability of 0.82, or 82%.

1.3 How Logistic Regression Works

The model follows three main steps:

Step 1: Calculate a linear combination

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Step 2: Convert it into a probability

$$\hat{y} = \sigma(z) = 1/(1 + e^{(-z)})$$

Step 3: Apply a decision threshold

$$\hat{y} \geq 0.5 \Rightarrow \text{Class } 1$$

$$\hat{y} < 0.5 \Rightarrow \text{Class } 0$$

Here:

- $x_1, x_2, \ldots, x_n$: input features.
- $w_1, w_2, \ldots, w_n$: weights learned during training.
- b: bias.
- z: linear combination of the inputs. ($z = wx+b$)
- $\hat{y}$: predicted probability of Class 1.
- $\sigma(z)$: sigmoid function.

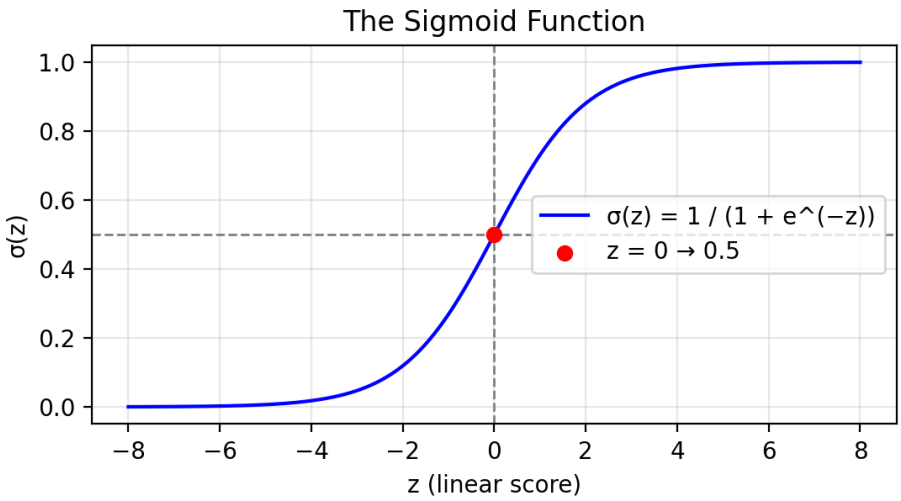
The threshold of 0.5 is a common default, not a rule that must always be used. It can be adjusted depending on the problem.

1.4 The Sigmoid Function

The sigmoid function converts any real number into a value between 0 and 1.

$$\sigma(z) = 1/(1 + e^{(-z)})$$

Here, e is Euler's number, approximately 2.71828.



z	Sigmoid σ(z)	Interpretation
-6	0.0025	Probability is nearly 0
-2	0.1192	Probability is low
0	0.5000	Probability is 50%
2	0.8808	Probability is high
6	0.9975	Probability is nearly 1

Important observations:

- If z is a large positive number, sigmoid approaches 1.
- If z is a large negative number, sigmoid approaches 0.

- If z = 0, sigmoid equals 0.5.
- The sigmoid output is a probability estimate, not the final class label.

1.5 Understanding the Decision Boundary

Suppose a model predicts whether a patient has diabetes.

- Predicted probability = 0.80 → Class 1.
- Predicted probability = 0.30 → Class 0.
- Predicted probability = 0.50 → Class 1 under the common `>= 0.5` rule.

For a model with one feature, the decision boundary occurs where:

$$\sigma(z) = 0.5$$

Because sigmoid equals 0.5 when z = 0, the boundary is:

$$wx + b = 0$$

For multiple features:

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = 0$$

This boundary is linear in the original feature space. With two features, it is generally a straight line; with three, a plane.

1.6 Advantages and Limitations

Advantages	Limitations
Simple to implement and interpret	Linear decision boundary unless features are transformed
Computationally efficient for many datasets	May struggle with complex nonlinear patterns & needs a larger dataset.
Produces probability estimates	Can be affected by outliers and influential observations
Provides interpretable feature coefficients	Performance depends on data quality and suitable features
Useful as a classification baseline	Scaling and regularization may be important, especially with differently scaled features

Note: Logistic Regression does not always require a very large dataset. Data requirements depend on the number of features, model complexity, noise, and how representative the training data is.

2. Mathematical Foundations of Logistic Regression

2.1 The Model Equation

Logistic Regression combines a linear equation with the sigmoid function.

First, calculate:

$$z = w^T x + b$$

Then calculate the probability:

$$\hat{y} = \sigma(w^T x + b)$$

For n input features:

$$\hat{y} = 1/(1 + e^{\wedge}(-(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)))$$

Here, $\hat{y}$ estimates the probability that the input belongs to Class 1.

2.2 Understanding Weights and Bias

Consider a simplified model that predicts whether a student passes an exam using study hours.

$$z = 1.2x - 5$$

Here:

- x = study hours.
- w = 1.2 = weight.
- b = −5 = bias.

If the student studies for 5 hours:

$$z = 1.2(5) - 5 = 1$$

The predicted probability becomes:

$$\hat{y} = 1/(1 + e^{-1}) \approx 0.731$$

Therefore, the model estimates a 73.1% probability of Class 1.

This is an illustrative example, not a scientifically validated estimate of passing probability.

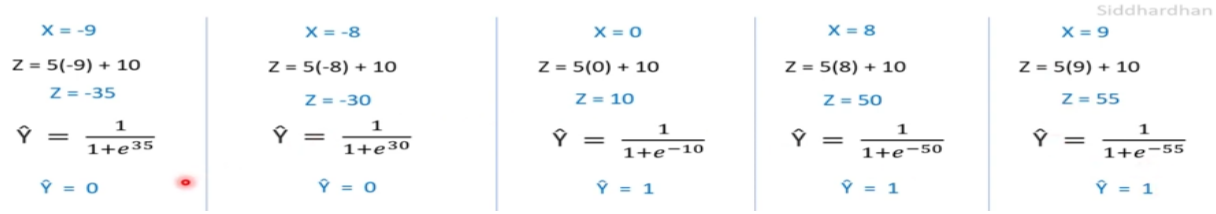
What do the parameters do?

- **Weight (w)** controls how an input feature affects the linear score.
- **Bias (b)** shifts the score and helps determine where the decision boundary lies.
- Training adjusts the weights and bias to reduce the model's prediction loss.

For n features, there are n weights and one bias.

2.3 Why Not Use Linear Regression for Classification?

$$Z = 5X + 10 \qquad \hat{Y} = \frac{1}{1+e^{-Z}}$$



Linear Regression can produce values below 0 or above 1. These values cannot directly represent probabilities.

For example, a linear model might predict −0.25 or 1.35.

Logistic Regression applies sigmoid to its linear score, restricting its output to the interval (0,1). This makes the output suitable for probability-based binary classification.

3. Loss Function and Cost Function

Training a model requires a way to measure how wrong its predictions are.

3.1 What Is a Loss Function?

A **loss function** measures the error for one training example.

For binary Logistic Regression, the standard choice is **Binary Cross-Entropy (BCE)**, also called **log loss**.

$$\mathbf{L(\hat{y},y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]}$$

Where:

- y = actual label, either 0 or 1.
- ŷ = predicted probability of Class 1. (from 0 to 1)
- log = natural logarithm.

The formula handles both possible true labels.

Case 1: Actual label y = 1

Substituting y = 1:

$$\mathbf{L = -\log(\hat{y})}$$

A high predicted probability for Class 1 gives a small loss. A low predicted probability gives a large loss.

Case 2: Actual label y = 0

Substituting y = 0:

$$\mathbf{L = -\log(1 - \hat{y})}$$

A low predicted probability for Class 1 gives a small loss. A high predicted probability gives a large loss.

3.2 Numerical Example

Suppose the actual label is y = 1.

Predicted probability $\hat{y}$	Loss $-\log(\hat{y})$	Meaning
0.90	0.105	Good prediction
0.70	0.357	Reasonably good
0.50	0.693	Uncertain prediction
0.10	2.303	Bad prediction
0.01	4.605	Very bad prediction

The model is penalized heavily when it confidently predicts the wrong class. This encourages it to produce probabilities that agree with the true labels.

3.3 What Is a Cost Function?

A **cost function** measures the average loss across the training dataset.

For m examples:

$$J(w,b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Here:

- m = number of training examples.
- i = index of a training example.
- $y^{(i)}$ = actual label for example i.
- $\hat{y}^{(i)}$ = predicted probability for example i.

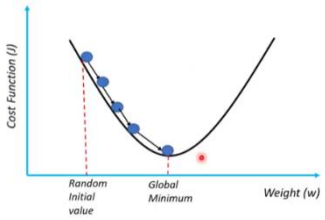
The model tries to find weights and bias that minimize this cost.

3.4 Loss vs. Cost

Loss Function	Cost Function
Measures error for one example	Usually averages loss across the dataset
Example: one patient's prediction	Example: predictions for all training patients
Used to assess individual predictions	Used as the overall training objective

Important: Binary Cross-Entropy is used because it is a suitable probabilistic loss for binary classification. For standard Logistic Regression with a linear score, its cost is convex in the model parameters, which makes optimization easier. BCE does not automatically make every possible machine learning model convex.

4. Gradient Descent for Logistic Regression



4.1 What Is Gradient Descent?

Gradient Descent is an optimization algorithm that adjusts model parameters to reduce the cost function.

Think of the cost as the height of a landscape. The model wants to move toward a lower point.

The process is:

1. Initialize the weights and bias, often to zero for basic Logistic Regression.
2. Calculate predicted probabilities using sigmoid.
3. Calculate the cost.
4. Calculate the gradients (partial derivatives).
5. Update weights and bias.
6. Repeat for many iterations or until a stopping condition is met.

4.2 The Six Core Equations

These equations form the mathematical foundation of Logistic Regression trained using batch Gradient Descent.

Equation 1: Linear score

$$z^{(i)} = w^T x^{(i)} + b$$

Equation 2: Sigmoid prediction

$$\hat{y}^{(i)} = 1/(1 + e^{\wedge}(-z^{(i)}))$$

Equation 3: Weight update

$$w \leftarrow w - \alpha \, \partial J/\partial w$$

Equation 4: Bias update

$$b \leftarrow b - \alpha \, \partial J/\partial b$$

Equation 5: Weight gradient

$$\partial J/\partial w = (1/m) \, X^T(\hat{y} - y)$$

Equation 6: Bias gradient

$$\partial J/\partial b = (1/m) \, \Sigma(i=1 \text{ to } m) \, (\hat{y}^{(i)} - y^{(i)})$$

Here, α is the learning rate, X is the feature matrix, and $\hat{y}$ is the vector of predicted probabilities.

The gradients indicate how the cost changes when the parameters change. The update subtracts the gradient to move toward lower cost.

4.3 Understanding the Weight Gradient

For one feature:

$$\partial J/\partial w = (1/m)\,\Sigma(i=1 \text{ to } m)\,(\hat{y}^{(i)} - y^{(i)})x^{(i)}$$

Each training example contributes:

(predicted probability − actual label)×feature value

The model averages these contributions over all training examples. The bias gradient is the average prediction error without multiplying by a feature value.

4.4 Learning Rate and Iterations

Term	Meaning
Learning rate (α)	Controls the size of each parameter update
Iteration	One Gradient Descent update using the full training dataset
Epoch	One complete pass through the training dataset
Convergence	The cost or parameters stop changing substantially

A learning rate that is too large may cause unstable updates or prevent convergence. A learning rate that is too small may make training very slow.

For standard, unregularized Logistic Regression with a linear score, the BCE objective is convex in the weights and bias. Gradient Descent can approach a global minimum with suitable optimization settings. However, convergence is not guaranteed for every learning rate or dataset, and a finite minimum may not exist for perfectly separable data without regularization.

5. Building Logistic Regression from Scratch Using NumPy

Now let's implement the algorithm ourselves, without using `sklearn.linear_model.LogisticRegression`.

The class will have four main methods:

- `__init__()` — sets the hyperparameters and initializes parameters.
- `fit()` — trains the model.
- `update_weights()` — calculates gradients and updates parameters.
- `predict()` — converts predicted probabilities into class labels.

We'll also create a `predict_proba()` method because probability estimates are useful for understanding model predictions.

5.1 Complete Implementation

```
import numpy as np


class LogisticRegression:
    def __init__(self, learning_rate=0.01, n_iters=1000):
        self.learning_rate = learning_rate
        self.n_iters = n_iters
        self.weights = None
        self.bias = 0.0

    def sigmoid(self, z):
        # Clip values to reduce the risk of exponential overflow.
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        # X: (number of samples, number of features)
        # y: (number of samples,)
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float).reshape(-1)

        if X.ndim != 2:
            raise ValueError("X must be a 2D feature matrix.")

        if len(X) != len(y):
            raise ValueError("X and y must have the same number of samples.")

        if not np.all(np.isin(y, [0, 1])):
            raise ValueError("y must contain only binary labels 0 and 1.")

        n_samples, n_features = X.shape

        # Initialize weights and bias.
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        # Batch Gradient Descent.
        for _ in range(self.n_iters):
            self.update_weights(X, y)

        return self

    def update_weights(self, X, y):
        n_samples = X.shape[0]

        # Step 1: Calculate linear scores.
        z = np.dot(X, self.weights) + self.bias

        # Step 2: Convert scores to probabilities.
        y_predicted = self.sigmoid(z)

        # Step 3: Calculate gradients.
        dw = np.dot(X.T, y_predicted - y) / n_samples
        db = np.sum(y_predicted - y) / n_samples
```

```
        # Step 4: Update weights and bias.
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

    def predict_proba(self, X):
        X = np.asarray(X, dtype=float)
        z = np.dot(X, self.weights) + self.bias
        return self.sigmoid(z)

    def predict(self, X, threshold=0.5):
        probabilities = self.predict_proba(X)
        return (probabilities >= threshold).astype(int)
```

5.2 Understanding the Code

Step 1 — Initialize the model

```
model = LogisticRegression(
    learning_rate=0.01,
    n_iters=1000
)
```

The learning rate controls the update size, while `n_iters` controls how many full-dataset updates are performed.

Step 2 — Train the model

```
model.fit(X_train, y_train)
```

During training, the model repeatedly calculates probabilities, computes gradients, and updates its weights and bias.

Step 3 — Obtain probabilities

```
probabilities = model.predict_proba(X_test)
```

For example, the output might be:

```
[0.12, 0.78, 0.43, 0.91]
```

These values are estimated probabilities of Class 1.

Step 4 — Obtain class predictions

```
predictions = model.predict(X_test)
```

Using the default threshold of 0.5, the result would be:

```
[0, 1, 0, 1]
```

5.3 Why Do We Use Matrix Operations?

Suppose there are m samples and n features.

Variable	Shape	Meaning
X	(m, n)	Feature matrix
y	(m,)	Actual labels

Variable	Shape	Meaning
`weights`	`(n,)`	One weight per feature
`np.dot(X, weights)`	`(m,)`	Linear score for each sample
`y_predicted`	`(m,)`	Predicted probability for each sample
`np.dot(X.T, y_predicted - y)`	`(n,)`	Gradient for each weight

The transpose `X.T` changes the shape of the feature matrix from `(m, n)` to `(n, m)`. This allows the gradient calculation to produce one gradient per feature.

Important: This implementation uses batch Gradient Descent and assumes that the features have already been appropriately preprocessed. It does not include regularization or feature scaling internally.

6. Applying the Model to the Pima Diabetes Dataset

The Pima Indians Diabetes dataset is a binary classification dataset commonly used to demonstrate diabetes prediction. The target is typically named `Outcome`:

- `Outcome = 0`: No diabetes.
- `Outcome = 1`: Diabetes.

The model uses patient measurements as input features and learns to predict the target.

6.1 Install the Required Libraries

Run this in your terminal or notebook if the packages are not already installed:

```
pip install numpy pandas scikit-learn
```

6.2 Load and Inspect the Dataset

This example assumes that you have a local CSV file named `diabetes.csv`, with the target column named `Outcome`.

```
import pandas as pd

df = pd.read_csv("diabetes.csv")

print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())
print(df["Outcome"].value_counts())
```

These commands help you understand the dataset, check missing values, and inspect the distribution of the target classes.

6.3 Separate Features and Target

```
X = df.drop(columns=["Outcome"]).to_numpy(dtype=float)
y = df["Outcome"].to_numpy(dtype=int)
```

Here:

- X contains all input features.
- y contains the binary target labels.

6.4 Split the Data and Scale the Features

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Why do we scale the features?

Features such as age, glucose level, and body mass index may have different numerical ranges. Scaling makes their magnitudes more comparable and can improve the behavior of Gradient Descent.

Important: Fit the scaler only on the training data. Then use that fitted scaler to transform the test data. This prevents information from the test set leaking into the preprocessing step.

6.5 Train the Custom Model

```
model = LogisticRegression(
    learning_rate=0.01,
    n_iters=5000
)

model.fit(X_train, y_train)
```

The model learns its weights and bias from the training dataset.

6.6 Evaluate the Model

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

The exact results depend on the dataset file and its preprocessing.

- **Accuracy:** Fraction of predictions that are correct.
- **Confusion matrix:** Shows true positives, true negatives, false positives, and false negatives.
- **Precision:** Among the samples predicted positive, how many were actually positive?
- **Recall:** Among all actual positive samples, how many did the model identify?
- **F1-score:** Harmonic mean of precision and recall.

For a medical prediction task, accuracy alone may be insufficient. Recall and false-negative counts can be especially important, depending on the intended use. A model like this is educational and should not be treated as a clinical diagnostic tool.

6.7 Save the Model Class in a Separate File

To reuse the model in another project, save the class from Section 5 in a file named `logistic_regression.py`.

Then, in your main script:

```
from logistic_regression import LogisticRegression

model = LogisticRegression(
    learning_rate=0.01,
    n_iters=5000
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

If you need predictions for new raw patient measurements, apply the **same fitted scaler** before calling `predict()`:

```
new_patient = [[
    6, 148, 72, 35, 0, 33.6, 0.627, 50
]]

new_patient_scaled = scaler.transform(new_patient)

probability = model.predict_proba(new_patient_scaled)
prediction = model.predict(new_patient_scaled)

print("Probability of Class 1:", probability[0])
print("Predicted class:", prediction[0])
```

The example assumes eight features in the same order as the CSV's feature columns. Replace the example values with appropriately formatted data from your actual dataset.

7. Machine Learning Interview Questions

Q1. What is Artificial Intelligence (AI)?

Answer: Artificial Intelligence is a **branch of computer science** concerned with **building systems that** can **perform tasks associated with human intelligence**, such as reasoning, learning, decision-making, and problem-solving.

Examples include virtual assistants, recommendation systems, and autonomous vehicles.

Q2. What is Machine Learning (ML)?

Answer: Machine Learning is **a field of AI** in which computers **learn patterns from data** to **make** predictions or **decisions without being explicitly programmed** with a separate rule **for every situation**.

For example, a model can learn from historical house prices to predict the price of a new house.

Q3. What is the relationship between AI, ML, and Deep Learning?

AI is the broad field, ML is a subset of AI, and Deep Learning is a subset of ML.

```
Artificial Intelligence (AI)
└── Machine Learning (ML)
    └── Deep Learning (DL)
```

Deep Learning sits within ML, which sits within the broader field of AI.

Q4. What is Deep Learning?

Answer: Deep Learning is a **subfield of Machine Learning** that **uses artificial neural networks** with multiple layers **to learn complex patterns from data**.

It is particularly useful for image recognition, speech recognition, natural language processing, and other tasks involving complex data.

Q5. What are the limitations of Deep Learning?

Answer:

- It often **requires large amounts of training data.**
- Training can **require substantial computational resources**, including GPUs or TPUs.
- Models can be **computationally expensive** and **time-consuming to train**.
- Designing, tuning, and interpreting deep neural networks can be **challenging**.

These are common limitations, not universal requirements for every deep learning model.

Q6. What is a Machine Learning model?

Answer: A Machine Learning model is a **mathematical function** or learned system that **captures patterns from training data** and uses them **to produce predictions for new inputs.**

For example, a Linear Regression model can be represented as:

$$\hat{y} = wx + b$$

Here, x is the input, w is the learned weight, b is the bias, and ŷ is the predicted output.

In Logistic Regression, the model applies a sigmoid function to its linear score to estimate a class probability.

Q7. How does a Machine Learning model learn?

Answer: A **model learns by using training data to adjust its internal parameters** so that its predictions improve according to a chosen objective or loss function.

For example, Linear Regression learns weights and bias to reduce prediction error. Logistic Regression learns its parameters to reduce classification loss, commonly Binary Cross-Entropy.

Q8. What are the main types of Machine Learning?

Type	Training information	Main purpose	Example
Supervised Learning	**Labeled data**	**Predict known targets**	**Diabetes classification**
Unsupervised Learning	**Unlabeled data**	**Discover patterns** or structure	**Customer clustering**
Reinforcement Learning	**Rewards from interactions**	**Learn actions** that **maximize cumulative reward**	**Game-playing agent**

Supervised Learning includes classification and regression.

Unsupervised Learning includes clustering and dimensionality reduction.

Reinforcement Learning involves an agent interacting with an environment, taking actions, and receiving rewards or penalties.

Q9. What are the two main types of Supervised Learning?

Answer:

1. Regression: Predicts a continuous numerical value.

Examples:

- House price prediction.
- Salary prediction.
- Temperature forecasting.

2. Classification: Predicts a discrete class or category.

Examples:

- Spam vs. not spam.
- Diabetes vs. no diabetes.
- Defective vs. non-defective product.

Regression	Classification
Predicts a numerical quantity	Predicts a class
Example: salary = 40,000	Example: class = 1
Linear Regression is a common algorithm	Logistic Regression is a common algorithm

Q10. What are the real-world applications of Machine Learning?

Answer:

- **Healthcare:** Medical-image analysis, risk prediction, and decision support.
- **Autonomous vehicles:** Sensor interpretation, object detection, and driving decision support.
- **Computer Vision:** Face recognition, object detection, image classification, and defect detection.
- **Natural Language Processing (NLP):** Chatbots, language translation, sentiment analysis, and text classification.

8. Quick Revision Sheet

Concept	Key point
Logistic Regression	A supervised classification algorithm
Binary classification	Predicts between two classes, often 0 and 1
Linear score	$z = w^T x + b$
Sigmoid	$\sigma(z) = 1/(1 + e^{(-z)})$
Predicted probability	$\hat{y} = \sigma(z)$
Default threshold	0.5
Loss function	Measures error for one example
Binary Cross-Entropy	Standard loss for binary Logistic Regression
Cost function	Average training loss
Gradient Descent	Updates parameters to reduce cost
Learning rate	Controls update size
Weights	One parameter per input feature
Bias	A single intercept parameter
`fit()`	Trains the model
`predict_proba()`	Returns estimated probabilities
`predict()`	Returns class labels
`StandardScaler`	Standardizes feature values
AI	Broad field of intelligent systems
ML	Learns patterns from data

Concept	Key point
Deep Learning	Uses multilayer neural networks
Supervised Learning	Uses labeled training data
Unsupervised Learning	Learns patterns from unlabeled data
Reinforcement Learning	Learns through actions and rewards

9. Self-Test: Check Your Understanding

Try answering these questions before checking the answers.

1. Which function converts the linear score into a probability in Logistic Regression?
2. What is the usual loss function for binary Logistic Regression?
3. What does the learning rate control?
4. What is the shape of `X` when there are 100 samples and 8 features?
5. Why should `StandardScaler` be fitted only on the training data?
6. Which task is an example of classification?
7. What is the correct relationship between AI, ML, and Deep Learning?

Answer Key

1. **Sigmoid function** — maps a real-valued score to a value between 0 and 1.
2. **Binary Cross-Entropy** — penalizes incorrect probability estimates for binary labels.
3. **The size of parameter updates** — it controls how far weights and bias move in each update.
4. **`(100, 8)`** — rows represent samples and columns represent features.
5. **To avoid data leakage** — fit on training data, then transform the test data using the same scaler.
6. **Predicting whether an email is spam** — it predicts a category.
7. **AI contains ML, and ML contains Deep Learning** — Deep Learning is a subset of ML, which is a subset of AI.